E4 234 Advanced Power System Analysis — Programming Assignment: AI-Assisted Load Flow Simulator Development

Send your files as a single compressed file to: e4234.course@gmail.com

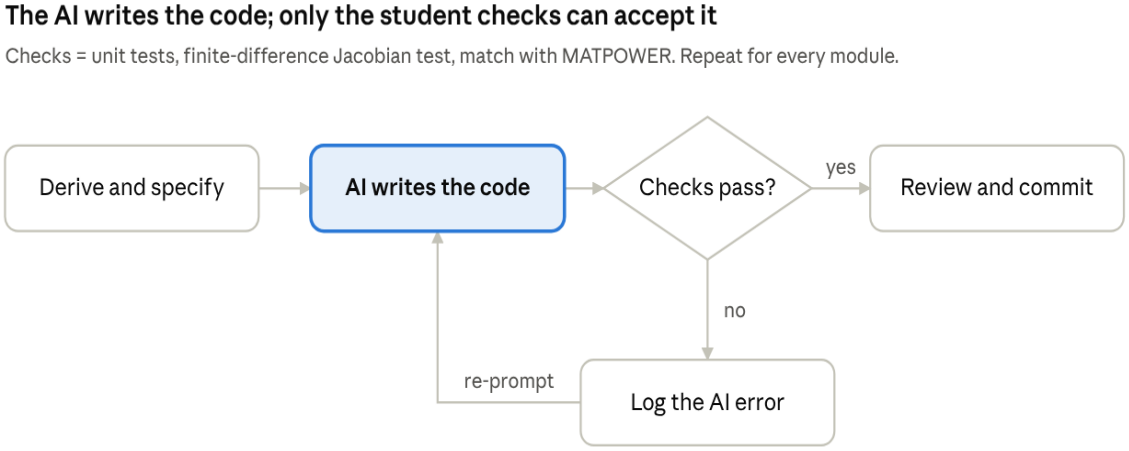
Deadline: 4th November 11:59 PM

1. Overview and learning objectives

Each student builds a desktop Newton-Raphson load flow simulator with an ETAP/PowerFactory-style single-line-diagram (SLD) GUI, **writing the code with AI coding assistants** (Claude, ChatGPT, GitHub Copilot, Cursor or similar). The AI writes the code; the student specifies it, verify it and answer for every line. The assignment carries **15 of the 100 course marks**.

By the end of the assignment, students should be able to:

1. Derive the Newton-Raphson load flow equations and Jacobian, and turn them into a specification precise enough for an AI assistant to implement.
2. Model transformers with off-nominal taps and phase shift, bus shunts, and generator reactive power limits.
3. Plan a software architecture that keeps network data, solver and visualisation separate, and direct an AI tool to build it module by module.
4. Verify AI-generated engineering code independently, with unit tests, a finite-difference Jacobian check and comparison against reference tools.
5. Recognise the typical errors AI tools make in power system code, and judge where these tools help and where they mislead.



2. Logistics

Individual assignment, Python 3.10 or later, AI coding assistants required, ending with a live demo and viva.

- **Individual work:** each student submits their own simulator. Discussing ideas is fine; sharing code, prompts or logs is not.
- **Language:** Python, so that all submissions run on one evaluation machine.
- **AI tools:** any coding assistant or AI-enabled editor, free or paid. Students without access to a paid tool can use free tiers; the marks do not depend on which tool is used.

Category	Allowed	Not allowed inside the simulator
Numerics	NumPy, SciPy (`scipy.sparse`, `spsolve`, `splu`)	Any ready-made power flow routine
Power system packages	MATPOWER, pandapower, PYPOWER **only** to produce reference results for validation	`pandapower.runpp`, `pypower.runpf` or MATPOWER calls in the solve path
GUI	PyQt6 / PySide6 (recommended: `QGraphicsScene`), Tkinter, or a web GUI (Dash, NiceGUI)	Wrapping another simulator's GUI
Plotting and reports	Matplotlib, Plotly, ReportLab, openpyxl	—

3. Part A — Newton-Raphson load flow engine

The engine solves the AC power flow equations to a maximum mismatch of 10^{-8} pu and must reproduce reference solutions on the IEEE test systems.

A1. Network modelling

1. Build the bus admittance matrix Ybus as a `scipy.sparse` matrix from branch data, never by dense inversion.
2. Lines: π-model with series R, X and total line charging B.
3. Transformers: off-nominal tap ratio and phase-shift angle on either side (MATPOWER branch model), giving an asymmetric Ybus for phase shifters.
4. Bus shunts (fixed G and B), constant-power loads, and at least one other load model (constant impedance or ZIP) as a user option.
5. Bus types slack, PV and PQ; multiple generators at one bus; out-of-service elements excluded without renumbering by hand.

A2. Newton-Raphson solver

1. Newton-Raphson in polar coordinates with the four Jacobian sub-matrices ($\partial P/\partial \theta$, $\partial P/\partial V$, $\partial Q/\partial \theta$, $\partial Q/\partial V$) derived analytically and assembled as a sparse matrix.
2. Each correction solved by sparse LU factorisation; no explicit Jacobian inverse.
3. Initial point: flat start, or a user-supplied V and θ vector (for example, the previous solution when rerunning a study).
4. User options: tolerance, maximum iterations, and Q-limit enforcement on or off.
5. Generator reactive limits: PV-to-PQ switching when Q hits a limit, and PQ-to-PV back-switching when the voltage recovers. Log every switch.

A3. Outputs

1. Bus V (pu, kV), θ (deg), P and Q injections, generator P and Q.
2. Branch P, Q, S flows at both ends, loading as a percentage of rating, and I^2R losses; total system losses.

A4. Study (for the 3-page report)

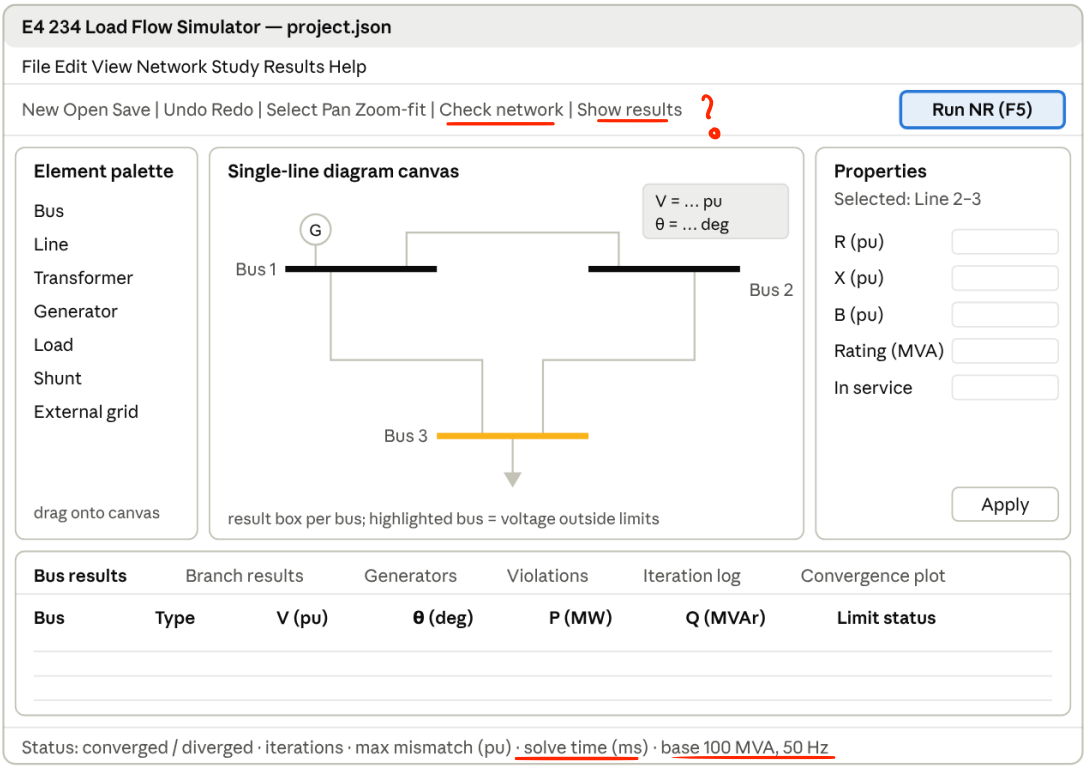
Test performance for the IEEE 14, 30, 57 and 118-bus systems, report NR iterations, total time and time per iteration.

4. Part B — Graphical user interface

The GUI must let a user who has never seen the code build a small network, run a load flow and read the results, as in commercial software such as ETAP or PowerFactory.

B0. Target layout

Target main window: palette left, diagram centre, properties right, results below



The main window follows the layout of ETAP and PowerFactory. Panels may be docked differently, but all seven regions must be present.

Region	Contents	Behaviour
Menu bar	File, Edit, View, Network, Study, Results, Help (see B4)	Every action reachable from a menu; shortcuts shown next to items
Toolbar	File actions, undo/redo, select/pan/zoom-to-fit, check network, **Run NR**, show results	Run is disabled while the network check fails

Region	Contents	Behaviour
Element palette	Bus, line, transformer, generator, load, shunt, external grid	Drag onto the canvas; branch elements connect by clicking two buses
Canvas	The single-line diagram, with result boxes after a run	Zoom, pan, snap to grid, multi-select, right-click menu (properties, delete, take out of service)
Properties panel	Fields of the selected element (see B5)	Edits validated on Apply; units shown; invalid values highlighted
Results dock	Tabs: bus results, branch results, generators, violations, iteration log, convergence plot	Sortable, filterable tables; clicking a row selects the element on the canvas
Status bar	Converged or diverged, iterations, max mismatch, solve time, system base	Updated after every run

B1. Single-line diagram editor

1. Element palette: bus (busbar), line, two-winding transformer, generator, load, shunt, and external grid (slack).
2. Drag-and-drop placement, connection by clicking two buses, move, delete, undo/redo (at least 10 steps), zoom and pan.
3. Double-click opens a property dialog with units shown (for example R in Ω or pu, rating in MVA) and input validation: negative ratings, dangling branches and islands without a slack bus are flagged before the run.
4. Imported cases (IEEE CDF, MATPOWER `.m`) get an automatic layout; a force-directed layout is acceptable. The user can then rearrange buses and save the layout.

B2. Running a study

1. A study-case panel: tolerance, maximum iterations, flat start or previous solution, Q-limit enforcement on or off, and global load scaling.
2. A progress/log window showing the iteration mismatch and any PV-PQ switching.

B3. Viewing results

1. Result boxes on the diagram next to each bus (V, θ) and branch (P, Q, loading %), which the user can toggle on or off.

2. Colour coding: buses outside their voltage limits (default 0.95–1.05 pu) and branches above 80% and 100% loading, with a legend.
3. Sortable bus, branch and generator result tables.
4. Export: results to CSV or Excel and a one-click PDF or HTML report with the diagram and summary tables.
5. Save and reopen a project file (network data, layout and study settings) in JSON.

B4. Menus and shortcuts

Menu	Items	Shortcuts
File	New, Open project, Save, Save as, Import IEEE CDF / MATPOWER, Export case as MATPOWER, Export results (CSV/XLSX), Export report (PDF/HTML), Recent files, Exit	Ctrl+N, Ctrl+O, Ctrl+S
Edit	Undo, Redo, Copy, Paste, Delete, Select all, Find element by name or number	Ctrl+Z, Ctrl+Y, Del, Ctrl+F
View	Zoom in, zoom out, fit to window; show/hide result boxes, colour coding, grid; reset panel layout	Ctrl++, Ctrl+−, F
Network	Check network (islands, missing slack, bad data), Auto-layout, System base MVA and frequency	Ctrl+K
Study	Load flow settings, Run NR, Global load scaling, Q-limit enforcement on/off	F5
Results	Bus, branch and generator tables; loss summary; violations list; convergence plot	—
Help	User guide, About (version, student name and roll number)	F1

B5. Element property fields

Every field shows its unit; data entered in physical units is converted to per unit on the system base (default 100 MVA, 50 Hz) and the pu value is shown alongside.

Element	Required fields
Bus	Number, name, nominal kV, Vmin and Vmax (pu), area; type (slack/PV/PQ) set from what is connected
Line	From and to bus, R, X, total charging B (pu, or Ω/km and µS/km with length in km), rating (MVA), in service
Transformer	HV and LV bus, rated MVA, rated kV on each side, impedance (%Z and X/R), tap ratio, tap side, tap range and step, phase shift (deg), in service
Generator	Bus, P set-point (MW), V set-point (pu), Qmin and Qmax (MVAr), in service
Load	Bus, P (MW), Q (MVAr) or power factor, model (constant PQ, constant Z or ZIP with coefficients), in service
Shunt	Bus, G (MW) and B (MVAr) at 1 pu voltage, in service
External grid	Bus, V set-point (pu), angle (deg)

B6. Result display conventions

1. Bus result box: V in pu to 3 decimals and θ in degrees to 2 decimals; a toggle shows V in kV instead.
2. Branch result box: P (MW) and Q (MVAr) at the sending end with an arrow showing flow direction, and loading in %.
3. Colour coding: undervoltage (below Vmin) and overvoltage (above Vmax) in two distinct colours; branches at 80–100% loading in amber and above 100% in red; out-of-service elements dashed grey. A legend is visible whenever colour coding is on.
4. Stale results: editing the network after a run greys out all results until the next run.
5. Convergence plot: $\log_{10}$ of the maximum mismatch against iteration number.
6. Divergence: a clear message with the last mismatch and the buses with the largest mismatch highlighted on the canvas.

7. Test systems and acceptance criteria

A submission passes only if its NR results match the reference solutions on all four IEEE systems within the tolerances below.

Check	Tolerance
Convergence: largest P or Q mismatch	≤ 10^{-8} pu
Bus voltage magnitude vs reference	≤ 10^{-4} pu
Bus voltage angle vs reference	≤ 0.01°
Branch P and Q flows vs reference	≤ 0.01 MW / MVAr
Total system losses vs reference	≤ 0.1%
Final NR iterations on IEEE 118 (flat start, Q limits off)	≤ 7

Reference results are produced with MATPOWER `runpf` or pandapower `runpp` using the same options (Q limits on or off, same tolerance). Each student submits a script `validate.py` that runs all four cases and prints a pass/fail table against these tolerances.

8. Deliverables and submission

One zip file per student, named `E4234_Name.zip`, uploaded to the course page before the deadline, and viva.

1. **Source code** in a Git repository (a private GitHub or GitLab link shared with the instructor and TAs), committed module by module. A repository with one large final commit loses the code-quality marks.
2. **README** with installation steps (`requirements.txt` or `environment.yml`), how to launch the GUI, and how to run `validate.py` and the test suite.
3. **Viva and live demo** for 10 minutes